

M1 — Baseline Web Application

Submission — Mini Shop (Option B, E-commerce)

CSCE 553 · Principles of Software Methodology · Fall 2026

Team: Sabri Kahoul, Chloe

This is a CSCE 553 class baseline, not a production system. All users and data are fake.

Submission links

Public app (HTTPS)	https://csce-500-project.onrender.com
Interactive API docs	https://csce-500-project.onrender.com/docs
GitHub repository	https://github.com/chloemich04/CSCE-500-Project

The repository README (full API table + setup) and this evidence are in the repo. The app is on Render's free tier; the first request after ~15 min idle may take about a minute (cold start).

1. Description

Mini Shop is a baseline e-commerce web application (a video-game shop). We chose **Option B — E-commerce**. Store managers create and edit products; customers search products, add them to a cart, and place orders. Email/password authentication works through the API and returns a JWT. All data is stored in PostgreSQL (Supabase).

2. Architecture

Browser (HTML pages + JS) → FastAPI application (JSON API + HTML), hosted on Render → PostgreSQL database, hosted on Supabase.

The browser stores the JWT returned at login and sends it on protected calls. The browser never talks to Supabase directly — only the API uses the database connection (DATABASE_URL).

3. Demo accounts

All three use the password **ClassDemo123!**

Email	Password	Account type
alex@example.com	ClassDemo123!	customer
blair@example.com	ClassDemo123!	customer
casey@example.com	ClassDemo123!	store_manager

4. HTTP API

Base URL: <https://csce-500-project.onrender.com> — JSON request/response bodies. Protected routes require Authorization: Bearer <token>.

Method	Path	Auth	Purpose
GET	/health	No	Service health check
POST	/api/auth/register	No	Create an account
POST	/api/auth/login	No	Log in; returns JWT
GET	/api/products	No	List products
GET	/api/products/search	No	Search products by text
POST	/api/products	Manager	Create a product
PUT	/api/products/{id}	Manager	Update a product
POST	/api/cart	Yes	Add product to cart
GET	/api/cart	Yes	View cart
PUT	/api/cart/{item_id}	Yes	Update cart quantity
DELETE	/api/cart/{item_id}	Yes	Remove cart item
POST	/api/orders	Yes	Checkout (create order)
GET	/api/orders	Yes	List user's orders

5. Evidence — real terminal output against the hosted URL

Commands run from PowerShell against <https://csce-500-project.onrender.com> (not localhost), captured after the service was awake.

Health check (read)

Login (returns a JWT)

```
PS C:\Users\Sabri\CSCE-500-Project> $login = @{ email = "alex@example.com"; password =  
"ClassDemo123!" } | ConvertTo-Json  
>> $token = (Invoke-RestMethod -Uri "https://csce-500-project.onrender.com/api/auth/login"  
-Method POST -ContentType "application/json" -Body $login).access_token  
>> $token  
  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2IiwiaWZwLmFwaWwiOiJhbGV4QGV4YW1wbGUyY29tIiwiaWF0IjE6NjM5ODg5OTUxLCJleHAiOjE3MDg5OTUxMTk5LlUicA7ZFzU2rhyr_LBEph3ikTbiG03ClnpyNDQ
```

Write — add a product to the cart

```
PS C:\Users\Sabri\CSCE-500-Project> $cart = @{ product_id = 1; quantity = 2 } | ConvertTo-Json
>> Invoke-RestMethod -Uri "https://csce-500-project.onrender.com/api/cart" -Method POST
-ContentType "application/json" -Body $cart -Headers @{ Authorization = "Bearer $token" }

cart_id items total
-----
3 {id=3; product_id=1; name=test; price=1,0; quantity=2; line_total=2,0} 2,0
```

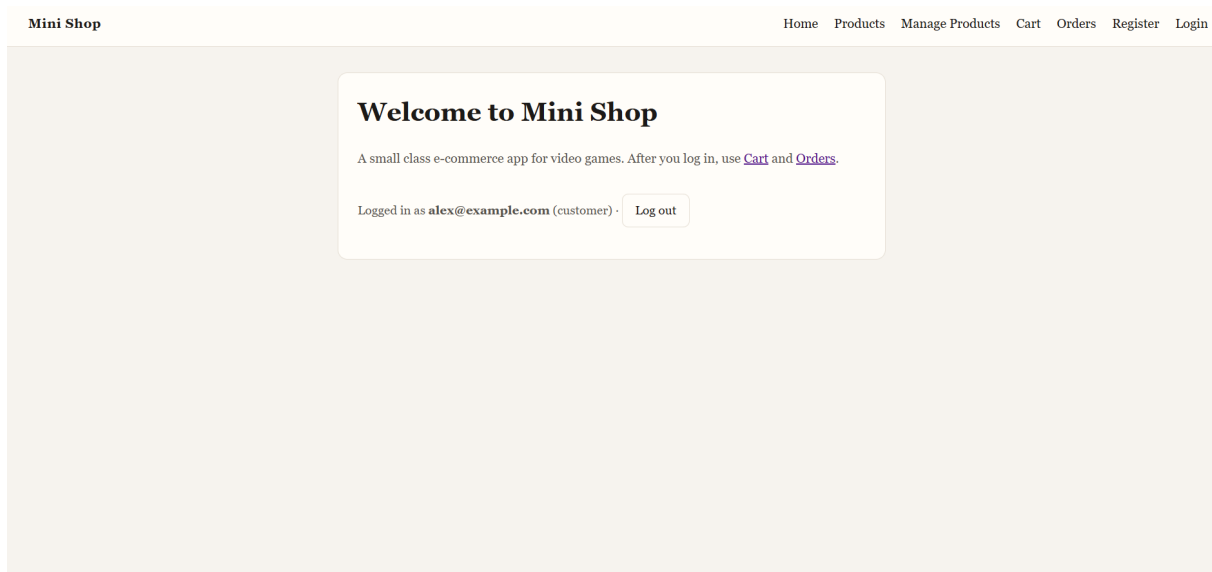
Read — view the cart

```
PS C:\Users\Sabri\CSCE-500-Project> Invoke-RestMethod -Uri
"https://csce-500-project.onrender.com/api/cart" -Headers @{ Authorization = "Bearer $token" }

cart_id items total
-----
3 {@fid=3; product_id=1; name=test; price=1,0; quantity=2; line_total=2,0} 2,0
```

6. UI screenshot (logged in)

The hosted app home page while logged in as alex@example.com (customer). The header shows "Logged in as alex@example.com (customer)" with a Log out button, confirming an authenticated session on the deployed app.



Screenshot: Mini Shop home page, logged in as alex@example.com on <https://csce-500-project.onrender.com>

7. Notes

- Email/password authentication works through the API; login returns a JWT sent as Authorization: Bearer <token> on protected routes.
- At least three demo users exist in the deployed system (alex@, blair@, casey@).
- Data persists in Supabase PostgreSQL; the browser never talks to the database directly.
- "Buy" creates an order row in the database — there is no real payment processing.
- Not a production system. Fake data only.